

GUIDANCE, NAVIGATION & CONTROL

Rendezvous and Docking maneuvers [EN]

Authors:

Francisco Palma (105949)	francisco.c.palma@tecnico.ulisboa.pt
Miguel Ervideira (106129)	miguel.s.ervideira@tecnico.ulisboa.pt
Rafael Ferreira (106146)	rafael.r.ferreira@tecnico.ulisboa.pt
Guilherme Tavares (106607)	guilherme.s.tavares@tecnico.ulisboa.pt
Pedro Dinis (106733)	pedro.goncalves.oliveira.dinis@tecnico.ulisboa.pt
Simone Binotto (106780)	simone.binotto.2@studenti.unipd.it
Tomás Casimiro (116269)	tomas.casimiro@tecnico.ulisboa.pt

2025/2026 – 2nd Semester

Contents

1	Introduction	3
1.1	Mission	3
1.2	Far-Field Rendezvous	3
1.3	Near-Field Rendezvous	3
1.4	Terminal Docking	3
2	Theory	4
2.1	Translational Dynamics	4
2.2	Angular Dynamics	4
2.3	Reference Frames	5
2.3.1	Reference Frames rotation	5
3	Spacecraft Model and Specifications	7
4	Far-Field Rendezvous	8
4.1	Approach Strategy	8
4.2	Dynamics	8
4.2.1	Lambert's Problem in the CW Frame	8
4.2.2	Position Error Model	9
4.2.3	Transfer Time Optimisation	9
4.2.4	Reference Trajectory Generation	9
4.2.5	Delta-V Budget	10
5	Near-Field Rendezvous	11
5.1	LQR Controller	11
5.1.1	Controller Design - LQR	12
5.1.2	Controller Implementation	13
5.2	Attitude controller	14
5.2.1	Controller Design - PD Controller	15
5.2.2	Reaction Control System (RCS)	15
5.3	Position Estimation - KF	17
5.3.1	Reference Frame and Navigation State	18
5.3.2	CW Process Model	18
5.3.3	Sensors	18
5.3.4	Kalman Filter	19
5.3.5	Sensor Manager and the <code>update_flag</code>	19
5.4	Attitude Estimation - MEKF	20
5.5	Sensors	20
5.5.1	Star Trackers	20
5.5.2	Gyroscopes	21
5.5.3	Laser Radar	22
5.6	MEKF	23
5.6.1	Measurement Update	23
5.6.2	Reset	24
5.6.3	Propagation	24

6	Terminal Docking	26
6.1	Docking Port Geometry	26
6.2	MPC Controller	26
6.2.1	Discrete-Time Model	26
6.2.2	Optimisation Problem	26
6.2.3	Actuator Limits and Scaling	27
6.2.4	Cost Function Weights	27
6.3	Waypoint Sequencer	27
6.4	Attitude Controller	28
6.4.1	PD Control Law	28
6.4.2	Pointing Reference	28
6.5	RCS During Terminal Docking	28
6.6	Docking Navigation	28
6.6.1	Camera Sensor	29
6.6.2	Filter Behaviour at Docking	29
6.6.3	Results	29
6.7	Simulation Results	30

1 Introduction

This work, developed as part of the Guidance Navigation and Control course taught by Professor Alan Souza, simulates a spacecraft rendezvous and docking mission in Low Earth Orbit (LEO).

Orbital rendezvous is the maneuver in which a chaser spacecraft closely approaches a target spacecraft. While it is a routine operation today, frequently used to transport cargo and astronauts to the International Space Station (ISS), the first successful rendezvous was pioneered in the 1960s to lay the critical groundwork for the Apollo lunar missions.

Docking constitutes the final stage of this process. It demands highly precise control to safely bring the two spacecraft together and physically lock them, enabling the transfer of crew and materials. Due to the strict physical tolerances and inherent risks involved, docking remains one of the most complex and critical phases of the overall mission.

1.1 Mission

After thorough consideration, the mission profile was divided into three distinct phases: Far-Field Rendezvous, Near-Field Rendezvous, and Terminal Docking. The Far-Field phase consists of open-loop maneuvers designed to bring the chaser into the general vicinity of the target. Once sufficiently close, the Near-Field phase engages a preliminary closed-loop control system to safely guide the chaser toward the docking axis. Finally, the Terminal Docking phase transitions to a highly precise, constraint-aware control strategy to manage the strict physical tolerances of the final approach and successfully secure the spacecraft.

1.2 Far-Field Rendezvous

At this stage of the mission, it is assumed that the chaser spacecraft has been successfully inserted into orbit and is already coplanar with the target. The primary objective of the Far-Field Rendezvous phase is to close the initial, large-scale distance between the two vehicles. This is achieved through orbital phasing, a process that alters the chaser's orbital period to allow it to shift its position along the orbit trajectory until its phase, or true anomaly, synchronizes with that of the target spacecraft.

1.3 Near-Field Rendezvous

During this phase, the chaser has reached close proximity to the target but is not yet correctly positioned for terminal docking. To safely bridge this remaining distance, a closed-loop control system, specifically a Linear Quadratic Regulator (LQR), is employed. This active controller manages the spacecraft's trajectory, smoothly guiding it into the immediate vicinity of the docking axis in preparation for the final approach.

1.4 Terminal Docking

During the Terminal Docking phase, the chaser transitions to a highly precise controller to execute the final approach and physically attach to the target. While the mechanical specifics of the physical connection are beyond the scope of this report, which focuses strictly on the Guidance, Navigation, and Control (GNC) system, the structural constraints and physical tolerances of the docking mechanism are fully accounted for within the control strategy.

2 Theory

Designing a Guidance, Navigation, and Control (GNC) system requires a physical model of the spacecraft. For simplicity, this work models the spacecraft as a rigid body, ignoring vibrations and energy losses like fuel sloshing or solar panel vibrations.

The spacecraft's motion can be separated into two parts: translational dynamics: the movement of its center of mass through space; and angular dynamics: its rotation around the center of mass. This natural decoupling simplifies the design of the GNC system, as shown in the following sections.

2.1 Translational Dynamics

Modeling translational dynamics as a two-body problem in an inertial frame yields the equation of motion:

$$\mathbf{F}_{net} = \mathbf{F}_g \implies m\ddot{\mathbf{r}} = -\frac{\mu m}{\|\mathbf{r}\|^3}\mathbf{r} \implies \ddot{\mathbf{r}} = -\frac{\mu}{\|\mathbf{r}\|^3}\mathbf{r}$$

where m is the spacecraft's mass, $\ddot{\mathbf{r}}$ is its acceleration, μ is the central body's standard gravitational parameter, and $\mathbf{r}$ is the position vector with norm $\|\mathbf{r}\|$.

The previous equation models a freely coasting spacecraft. To actively control the trajectory, we introduce a thrust force, $\mathbf{T}$:

$$\ddot{\mathbf{r}} = -\frac{\mu}{\|\mathbf{r}\|^3}\mathbf{r} + \frac{\mathbf{T}}{m}$$

This updated equation defines the spacecraft's controlled motion, providing the foundation for both the simulation and the control system design.

These equations approximate reality. We model the Earth as a perfect sphere, neglecting the J_2 perturbation caused by its oblateness, and ignore gravitational effects from the Sun and Moon.

2.2 Angular Dynamics

Angular dynamics are governed by Euler's equations of motion:

$$\boldsymbol{\tau} = \mathbf{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})$$

where $\boldsymbol{\tau}$ is the net external torque, $\mathbf{J}$ is the spacecraft's inertia matrix, $\dot{\boldsymbol{\omega}}$ is its angular acceleration, and $\boldsymbol{\omega}$ is its angular velocity.

To track the spacecraft's orientation, we represent its attitude using quaternions. We found this to be the most practical approach for our simulation. The quaternion kinematic equation is:

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \boldsymbol{\omega}$$

where $\dot{\mathbf{q}}$ is the quaternion rate of change, $\mathbf{q}$ is the current attitude quaternion, $\otimes$ denotes quaternion multiplication, and $\boldsymbol{\omega}$ is the angular velocity vector.

Similar to the translational dynamics, the previous equation models a freely rotating spacecraft. To actively control its orientation, we introduce a control torque, $\boldsymbol{\tau}_{ctrl}$:

$$\mathbf{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}) = \boldsymbol{\tau} + \boldsymbol{\tau}_{ctrl}$$

2.3 Reference Frames

This work utilizes three distinct reference frames depending on the specific application. The Body Frame is centered at the chaser's center of mass, with its axes aligned to the spacecraft's physical geometry. The Inertial Frame corresponds to the Earth-Centered Inertial (ECI) frame, with its origin at the Earth's geometric center. Finally, the Local Vertical, Local Horizontal (LVLH) frame is centered on the target spacecraft; its x -axis ($\bar{\mathbf{R}}$) points radially outward, its z -axis ($\bar{\mathbf{H}}$) aligns with the orbital angular momentum, and its y -axis ($\bar{\mathbf{V}}$) completes the right-handed coordinate system.

The Inertial frame, I , is primarily used to propagate the spacecraft's orbit. The Body frame, B , is essential because most onboard sensors output their measurements relative to the spacecraft itself. Finally, the LVLH frame is critical for relative navigation during rendezvous and docking operations. Since the overall mission, observers, and controllers rely on all three, we must define mathematical transformations to accurately convert measurements and quantities between these reference frames.

2.3.1 Reference Frames rotation

Although multiple methods exist to express rotations between reference frames, such as Euler angles or axis-angle representations, this work uses a combination of rotation matrices and quaternions. Overall quaternions are the go to in ADCS since they have no singularities.

A quaternion is a four-dimensional vector consisting of a scalar part, s , and a vector part, $\mathbf{v}$, expressed as $\mathbf{q} = [s, \mathbf{v}^T]^T$. To represent a rotation by an angle θ around a unit axis $\mathbf{u}$, the quaternion is defined as:

$$\mathbf{q} = \begin{bmatrix} \cos(\frac{\theta}{2}) \\ \mathbf{u} \sin(\frac{\theta}{2}) \end{bmatrix}$$

To represent a physical rotation, a quaternion must be a unit quaternion, meaning its norm is exactly one ($\|\mathbf{q}\| = 1$). To rotate an arbitrary vector $\mathbf{p}$ from the Body frame, B , to the Inertial frame, I , we use quaternion multiplication, denoted by the $\otimes$ symbol, alongside the quaternion conjugate, denoted by $*$. This rotation is expressed as:

$$\mathbf{p}^I = \mathbf{q}_B^I \otimes \mathbf{p}^B \otimes (\mathbf{q}_B^I)^*$$

where $\mathbf{p}^I$ is the vector expressed in the Inertial frame, $\mathbf{q}_B^I$ is the unit quaternion defining the rotation from the Body to the Inertial frame, $\mathbf{p}^B$ is the original vector in the Body frame, and $(\mathbf{q}_B^I)^*$ is the conjugate of the rotation quaternion.

It is often more straightforward to compute the rotation between the Inertial and LVLH frames using rotation matrices. The primary drawback of rotation matrices is their strict mathematical constraints: they must be orthogonal ($\mathbf{Q}\mathbf{Q}^T = \mathbf{I}$) and have a determinant of exactly one. To construct the rotation matrix from the Inertial frame to the LVLH frame, denoted as $\mathbf{Q}_I^{LVLH}$, we use the target spacecraft's inertial position ($\mathbf{r}_t$) and velocity ($\mathbf{v}_t$). We first calculate the directional unit vectors, where $\hat{\mathbf{x}}$ aligns with the position, $\hat{\mathbf{z}}$ aligns with the angular momentum, and $\hat{\mathbf{y}}$ completes the orthogonal triad. The final rotation matrix is formed by stacking these transposed unit vectors as rows:

$$\hat{\mathbf{x}} = \frac{\mathbf{r}_t}{\|\mathbf{r}_t\|}, \quad \hat{\mathbf{z}} = \frac{\mathbf{r}_t \times \mathbf{v}_t}{\|\mathbf{r}_t \times \mathbf{v}_t\|}, \quad \hat{\mathbf{y}} = \hat{\mathbf{z}} \times \hat{\mathbf{x}} \implies \mathbf{Q}_I^{LVLH} = \begin{bmatrix} \hat{\mathbf{x}}^T \\ \hat{\mathbf{y}}^T \\ \hat{\mathbf{z}}^T \end{bmatrix}$$

To seamlessly integrate this rotation matrix into the rest of the workflow, it must be converted into a quaternion. This transformation is accomplished using Shepperd's algorithm, a

numerically robust method that avoids singularities (division by zero) by evaluating the trace and diagonal elements of the rotation matrix, $\mathbf{Q}$. For the primary case where the trace is positive, a temporary variable K is used to compute the quaternion $\mathbf{q}$:

$$K = 2\sqrt{1 + \text{tr}(\mathbf{Q})}, \quad \mathbf{q} = \begin{bmatrix} K/4 \\ (Q_{32} - Q_{23})/K \\ (Q_{13} - Q_{31})/K \\ (Q_{21} - Q_{12})/K \end{bmatrix}$$

These methods will be used in subsequent sections to precisely rotate quantities between the established reference frames.

3 Spacecraft Model and Specifications

For the purposes of this mission simulation, the chaser spacecraft is represented using a simplified rigid-body dynamic model. This level of abstraction minimizes unnecessary computational complexity while retaining the core physical characteristics required to effectively design, tune, and validate the closed-loop control architecture. The vehicle has a total mass of 10,000 kg and is assumed to be symmetric across its primary axes, with rotational dynamics governed by the principal moment of inertia matrix, defined as $\mathbf{J} = \text{diag}(2.9, 2.9, 1.6) \times 10^4 \text{ kg} \cdot \text{m}^2$.

To successfully execute the rendezvous and docking phases, the vehicle is equipped with a comprehensive suite of navigational sensors designed to provide full-state observability across different flight regimes. This includes a Global Positioning System (GPS) and an Inertial Measurement Unit (IMU) for absolute orbital state estimation, a Star Tracker and Gyroscope for precise attitude determination, and a combined LIDAR and Camera system dedicated to tracking relative kinematics during proximity operations.

Control authority is provided by a dual-actuator propulsion setup. A main engine is utilized for large-scale orbital maneuvers (such as the far-field approach), while a Reaction Control System (RCS) is employed for precision attitude regulation and fine translational corrections. While the fidelity of individual component models varies, this overall configuration provides a highly realistic, manageable platform to fully achieve the mission objectives.

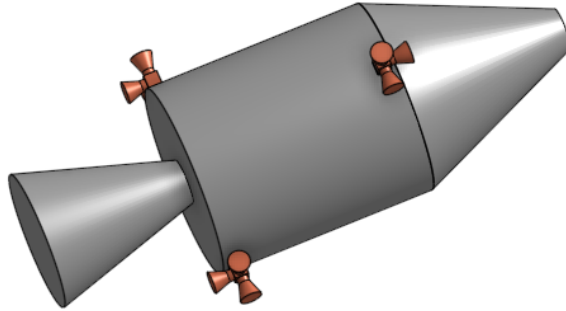


Figure 1: Conceptual model of the chaser spacecraft.

4 Far-Field Rendezvous

The Far-Field Rendezvous phase is responsible for generating an optimal open-loop reference trajectory from the chaser’s initial position to the vicinity of the target, together with a timed sequence of impulsive manoeuvres required to follow it. This trajectory is pre-computed before execution and exported to the workspace, where it serves as the reference for the closed-loop controllers of Sections ?? and 6.6.

The term *open-loop* refers to the fact that the manoeuvre sequence is planned in advance without real-time feedback: the guidance module computes where the chaser should be at every second of the mission, and the controllers in subsequent phases are responsible for tracking that reference.

4.1 Approach Strategy

Following the phased rendezvous strategy of Fehse [1] (§5.7), the full transfer is divided into three sequential sub-phases with intermediate hold points at decreasing distances from the target. The rationale for this structure is that navigation errors accumulate over time along any free-drift arc: the longer the arc, the larger the position error at its end. Inserting intermediate correction burns bounds this error by restarting the accumulation at each node.

The four waypoints are defined in the LVLH frame (Section 2.3) as follows:

Table 1: LVLH waypoint definitions.

Point	Position [m]	Velocity [m/s]	Description
S_0	[2000, −10000, 500]	[0, 0, 0]	Chaser at end of phasing
S_1	[0, −3000, 0]	[0, 0, 0]	Target orbit entry, 3 km behind
S_2	[0, −500, 0]	[0, 0, 0]	Hold point, 500 m behind
Dock	[0, 0, 0]	[0, 0, 0]	Docking port (LVLH origin)

Note that S_0 includes an out-of-plane offset of $z = 500$ m. This is handled correctly by the CW equations since the z -axis dynamics are fully decoupled from the in-plane motion, and the offset naturally decays over the course of the far-range transfer.

The three sub-phases and their configurations are summarised in Table 2.

Table 2: Far-Field Rendezvous sub-phase overview.

Sub-phase	Transfer	Distance	Max. impulses	Mid-course burns
Far-range	$S_0 \rightarrow S_1$	~10 km	4	up to 2
Close-range	$S_1 \rightarrow S_2$	2.5 km	3	up to 1
Final approach	$S_2 \rightarrow \text{Dock}$	500 m	2	0

The number of allowed mid-course corrections decreases with distance: the far-range sub-phase covers the largest separation and therefore permits up to two correction burns, while the final approach is short enough that a classic 2-impulse transfer suffices.

4.2 Dynamics

4.2.1 Lambert’s Problem in the CW Frame

The core computation of the guidance module is a boundary value problem: given the chaser’s current position $\mathbf{r}_0$ and a target position $\mathbf{r}_f$ to be reached after free drift for time Δt ,

find the required post-burn velocity $\mathbf{v}^+$. Using the CW State Transition Matrix introduced in Section 5.4.2:

$$\mathbf{r}_f = \Phi_{rr} \mathbf{r}_0 + \Phi_{rv} \mathbf{v}^+ \quad (1)$$

Solving for $\mathbf{v}^+$, provided $\det(\Phi_{rv}) \neq 0$:

$$\mathbf{v}^+ = \Phi_{rv}^{-1} (\mathbf{r}_f - \Phi_{rr} \mathbf{r}_0) \quad (2)$$

The required delta-V at that node is:

$$\Delta \mathbf{V} = \mathbf{v}^+ - \mathbf{v}_{\text{current}} \quad (3)$$

This calculation is repeated at every impulse node across all three sub-phases, implemented in `transfer_time_optimizer.m`. A singularity in Φ_{rv} occurs at integer multiples of the orbital period ($t = k T_{\text{orbit}}$); such transfer times are detected and discarded as infeasible.

4.2.2 Position Error Model

Following Fehse [1] (§4.4), the estimated final position error for a transfer with N_{segs} free-drift arcs of equal duration Δt_{seg} is modelled as:

$$e_1 = C_{\text{nav}} \Delta t_{\text{seg}} + C_{\text{thr}} \quad (4)$$

$$e_k = \frac{e_{k-1}}{2} + C_{\text{nav}} \Delta t_{\text{seg}} + C_{\text{thr}}, \quad k > 1 \quad (5)$$

where $C_{\text{nav}} = 0.05 \text{ m/s}$ is the navigation error growth rate per second of free drift, $C_{\text{thr}} = 0.20 \text{ m}$ is the position error introduced per impulsive burn, and $\Delta t_{\text{seg}} = t_{\text{transfer}}/N_{\text{segs}}$. Each mid-course correction halves the accumulated error from the previous arc, at the cost of one additional burn.

4.2.3 Transfer Time Optimisation

For each sub-phase, the optimal transfer time and number of impulses are selected by minimising the composite cost function:

$$J = 0.50 \cdot \frac{\Delta V_{\text{total}}}{\Delta V_{\text{ref}}} + 0.20 \cdot \frac{t_{\text{transfer}}}{T_{\text{orbit}}} + 0.30 \cdot \frac{e_{\text{final}}}{e_{\text{ref}}} \quad (6)$$

where the three terms penalise propellant consumption, transfer time, and estimated final position error, with weights reflecting mission priorities: propellant efficiency is weighted most (50%), followed by accuracy (30%) and time (20%).

The optimiser sweeps $N_{\text{mid}} \in \{0, 1, \dots, N_{\text{mid,max}}\}$ and, for each value, evaluates 300 candidate transfer times uniformly distributed in $[0.1 T_{\text{orbit}}, 1.5 T_{\text{orbit}}]$. For each candidate, Equation (2) is applied at every node to compute the delta-V sequence, and J is evaluated using the error model of Equations (4)–(5). The $(N_{\text{mid}}, t_{\text{transfer}})$ pair with minimum J is selected, implemented in `transfer_time_optimizer.m`.

4.2.4 Reference Trajectory Generation

Once the optimal manoeuvre sequence is determined for each sub-phase, the reference trajectory is generated second-by-second in `reference_trajectory.m`. For each free-drift arc, the departure delta-V is applied and the CW STM is evaluated at every integer second:

$$\mathbf{r}(t_0 + k) = \Phi_{rr}(k) \mathbf{r}_{\text{node}} + \Phi_{rv}(k) \mathbf{v}^+, \quad k = 0, 1, 2, \dots \quad (7)$$

At the end of each sub-phase, a 100-second station-keeping hold is appended at the waypoint, during which the reference position and velocity remain constant at $(\mathbf{r}_f, \mathbf{v}_f)$. This hold provides a window for the navigation filter to converge before the next sub-phase begins.

The three sub-phase trajectories are concatenated with correct time offsets into a single continuous reference $\{t_k, \mathbf{r}_k, \mathbf{v}_k\}$, exported to the workspace as `ref_trajectory` and consumed by the controllers of Sections ?? and 6.6.

4.2.5 Delta-V Budget

The total propellant cost is the sum of all impulse magnitudes across all three sub-phases:

$$\Delta V_{\text{total}} = \sum_{i=1}^{N_{\text{imp}}} \|\Delta \mathbf{V}_i\| \quad (8)$$

The exact values depend on the orbital parameters defined at runtime and are printed to the MATLAB console by `std_run_guidance.m`. A representative budget for an ISS-altitude orbit ($h = 400$ km, $n \approx 1.132 \times 10^{-3}$ rad/s, consistent with Section 5.4.1) is shown in Table 3.

Table 3: Representative delta-V budget.

Sub-phase	Impulses	ΔV [m/s]
Far-range	up to 4	(output of optimiser)
Close-range	up to 3	(output of optimiser)
Final approach	2	(output of optimiser)
Total		(output of optimiser)

5 Near-Field Rendezvous

The Near-Field Rendezvous phase bridges the critical gap between the initial orbital phasing and the terminal docking maneuvers. During this stage, the primary objective is to autonomously guide the chaser spacecraft from a far-field standoff distance into the immediate vicinity of the target, relying entirely on a closed-loop control architecture to manage the approach trajectory.

To validate the controller design, the simulation environment initializes the 10,000 kg chaser spacecraft in a standard Low Earth Orbit (LEO) at an altitude of 395 km ($R_E = 6371$ km, $\mu = 3.986 \times 10^{14}$ m³/s²). At the onset of this phase, the target is established in a nominal 400 km orbit. Relative to the target, the chaser is initially positioned 5 km behind along the orbital track and 5 km below in altitude. To accurately seed the simulation, this along-track separation ($\Delta\theta$) is translated into absolute Cartesian position and velocity vectors based on the nominal circular orbital dynamics.

The specific goal of the near-field controller is to drive the chaser from these initial dynamic conditions to a stable, terminal holding position defined as $\mathbf{x}_{ref} = [-150 \ 0 \ 0]^T$ m. Furthermore, because proximity operations inherently carry a high risk of collision, rigorous safety constraints must be maintained. Throughout the approach, a spherical keep-out zone with a strict 100-meter radius is enforced around the target spacecraft, ensuring the chaser maintains a safe standoff distance before clearance is granted for terminal docking.

5.1 LQR Controller

As previously mentioned, the mission relies on two controllers: a Linear Quadratic Regulator (LQR) and a Model Predictive Controller (MPC). The LQR handles the initial approach, bringing the chaser close enough to the target to safely engage the highly precise MPC for final docking.

The LQR is an optimal controller defined by the control law:

$$\mathbf{u} = -\mathbf{K}\mathbf{x}$$

where $\mathbf{u}$ is the control input, $\mathbf{K}$ is the feedback gain matrix, and $\mathbf{x}$ is the state vector.

The gain $\mathbf{K}$ is calculated to minimize the cost function:

$$J = \int_0^\infty (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}) dt$$

which is solved analytically using the Algebraic Riccati Equation [2].

The weighting matrices, $\mathbf{Q}$ and $\mathbf{R}$, dictate the controller's performance: increasing $\mathbf{Q}$ penalizes state errors, forcing the controller to track the desired trajectory more strictly; increasing $\mathbf{R}$ penalizes control effort, limiting actuator usage to conserve energy or fuel.

Balancing these weights is a core design trade-off, tuned iteratively through testing to meet mission requirements.

Furthermore, the Near-Field Rendezvous phase requires the final position reference to be established at a specific hold-point, defined as $\mathbf{x}_{ref} = [-150 \ 0 \ 0 \ 0 \ 0 \ 0]^T$. However, this state does not represent a natural equilibrium point of the unforced orbital dynamics.

Because standard Linear Quadratic Regulation (LQR) is designed to drive system states to the origin, the control architecture must be modified to accommodate non-zero setpoint tracking. To achieve this, the standard regulator is augmented with a steady-state feedforward control input, $\mathbf{u}_{ref}$, and applied to the state error. The modified control law becomes:

$$\mathbf{u} = \mathbf{u}_{ref} - \mathbf{K}(\mathbf{x} - \mathbf{x}_{ref})$$

The required feedforward control effort can be analytically estimated by evaluating the system dynamics at the desired reference point. By assuming steady-state conditions where the state derivative is zero ($\dot{\mathbf{x}} = \mathbf{0}$), we evaluate the linear state-space equation:

$$\mathbf{0} = \mathbf{A}\mathbf{x}_{ref} + \mathbf{B}\mathbf{u}_{ref}$$

This allows us to solve for the necessary feedforward input using the Moore-Penrose pseudoinverse of the input matrix, denoted as $\mathbf{B}^\dagger$:

$$\mathbf{u}_{ref} = -\mathbf{B}^\dagger \mathbf{A}\mathbf{x}_{ref}$$

Because $\mathbf{x}_{ref}$ is not a true dynamic equilibrium point, an exact solution that completely nullifies the state derivative does not exist. Instead, the pseudoinverse yields a least-squares approximation of the required control effort. As a result, the closed-loop system will not settle exactly on $\mathbf{x}_{ref}$, but rather converge to a stable equilibrium in its immediate vicinity. Given the operational parameters of the Near-Field approach, this minor steady-state tracking offset is considered acceptable and falls well within the required positional tolerances for the mission.

5.1.1 Controller Design - LQR

To design the LQR, we must first define the system in state-space form:

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \\ \mathbf{y} = \mathbf{C}\mathbf{x} \end{cases}$$

where $\mathbf{x}$ and $\mathbf{u}$ are the state and control input vectors, $\mathbf{A}$ and $\mathbf{B}$ are the system and input matrices, and $\mathbf{y}$ and $\mathbf{C}$ define the output.

The previously derived dynamics are nonlinear due to the $\|\mathbf{r}\|^3$ term, making them incompatible with the linear LQR method. Fortunately, we can use the Clohessy-Wiltshire (CW) equations [3]. These equations provide a linear approximation for the relative motion of a chaser and a target in circular orbits.

The reference frame is centered on the target body. The x -axis points radially outward, the z -axis aligns with the target's orbital angular momentum, and the y -axis completes the right-handed triad along the orbit track. Using the state vector $\mathbf{x} = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^T$, the linearized system matrices are:

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 3n^2 & 0 & 0 & 0 & 2n & 0 \\ 0 & 0 & 0 & -2n & 0 & 0 \\ 0 & 0 & -n^2 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

where n is the orbital rate (mean motion) of the target body.

For now, we assume full knowledge of the spacecraft's state. Later sections will design filters to estimate these states from sensor measurements.

The optimal gain matrix, $\mathbf{K}$, is calculated using MATLAB's `lqr` function, which numerically solves the Algebraic Riccati Equation. Although the LQR formulation mathematically guarantees a stable closed-loop system, verifying the closed-loop poles remains a recommended best practice.

5.1.2 Controller Implementation

With the gain matrix $\mathbf{K}$ computed, the control law is $\mathbf{u} = -\mathbf{K}\mathbf{x}$. Because $\mathbf{u}$ represents the required force per unit mass ($\mathbf{T}/m$), the actual thrust commanded to the engines is:

$$\mathbf{T} = m\mathbf{u}$$

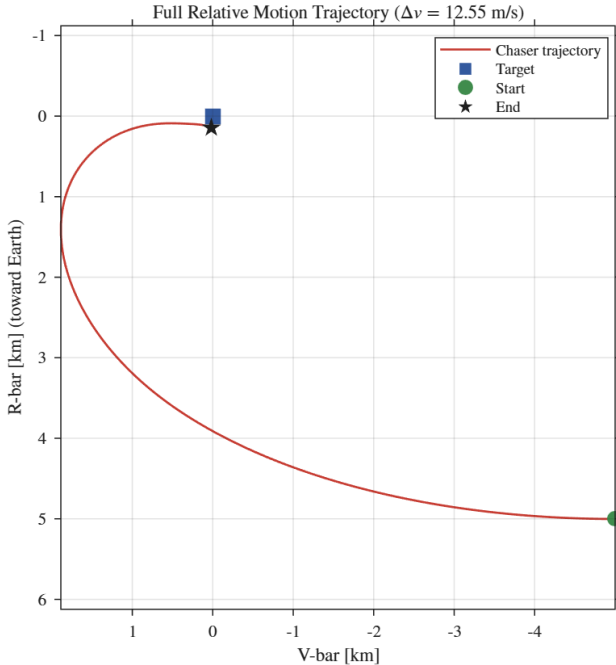
It is important to note that unlike the MPC, the LQR does not inherently account for physical constraints. Consequently, the commanded thrust may exceed actual hardware limits, such as saturation or slew rates.

To ensure physical realism and respect hardware limitations during the approach, an actuator saturation limit of 50 N is imposed on the propulsion system. The spacecraft is modeled with six directional thrusters, providing independent positive and negative control authority along all three body axes.

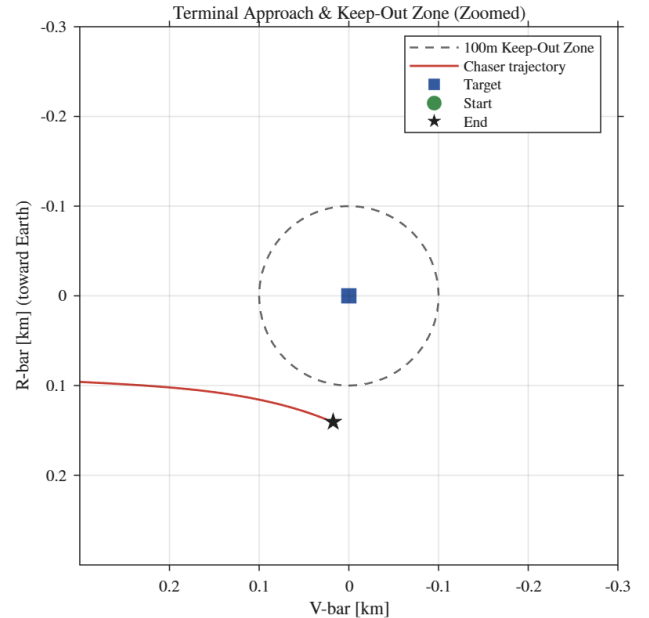
Through an iterative tuning process, the state and control penalty matrices ($\mathbf{Q}$ and $\mathbf{R}$) were optimized to balance tracking performance with fuel expenditure. The position states were penalized lightly, while the velocity states were heavily penalized to guarantee a slow, controlled approach and prevent overshoot. The finalized tuning matrices are defined as:

$$\mathbf{Q} = \text{diag}(1.25 \times 10^{-3}, 1.25 \times 10^{-3}, 1.25 \times 10^{-3}, 10^3, 10^3, 10^3), \quad \mathbf{R} = 5\mathbf{I}_{3 \times 3} = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 5 \end{bmatrix}$$

With these tuning parameters established, the closed-loop system response was simulated. The resulting spatial trajectory, relative distance, relative velocity, and commanded thruster forces are presented in the figures below.



(a) 3D approach trajectory of the chaser.



(b) Commanded control effort (50 N saturation).

Figure 2: Spatial trajectory and commanded thruster forces under LQR control.

As illustrated in the simulation results, the LQR controller successfully guides the chaser along a safe and controlled approach trajectory. As anticipated, the final position exhibits a minor steady-state tracking offset from the exact reference coordinates due to the non-equilibrium

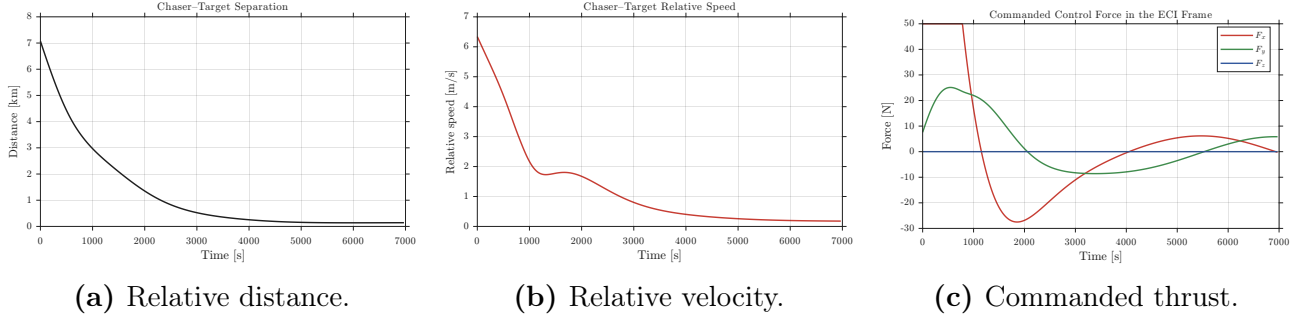


Figure 3: Relative kinematics and commanded control effort over the duration of the approach.

nature of the setpoint discussed previously. Nevertheless, the spacecraft stably settles in the immediate vicinity of the target without ever violating the 100-meter keep-out zone, fully satisfying the critical safety constraints of the mission.

The relative kinematics demonstrate highly desirable behavior. The relative distance exhibits a smooth, exponential decay, indicating a well-tuned, critically damped system response with absolutely zero spatial overshoot. The entire near-field maneuver is executed in approximately 6000 seconds, less than a single orbital period. Furthermore, the total mission fuel expenditure for this phase is highly efficient, consuming approximately 12.5 m/s of Δv , which aligns perfectly with standard operational budgets for real-world orbital rendezvous missions.

Examining the commanded control effort, the system behaves exactly as intended under significant initial error. The thrusters immediately hit their 50 N saturation limit at the onset of the maneuver to arrest the initial 5 km along-track separation. As the state errors naturally decrease along the trajectory, the control demands organically desaturate, allowing the continuous proportional feedback to smoothly and precisely steer the spacecraft into its final holding state.

Despite these excellent tracking dynamics, a fundamental limitation of the R-bar approach must be acknowledged. Because the final holding position on the R-bar is not a natural Keplerian equilibrium point, the system requires continuous actuation (and thus, continuous Δv expenditure) simply to maintain its position against natural orbital drift. To mitigate this penalty and prevent unnecessary propellant depletion, mission operations dictate that the spacecraft should minimize its dwell time at this setpoint and transition directly to the terminal docking phase as promptly as safety protocols allow.

5.2 Attitude controller

Attitude control is a critical subsystem in space missions, enabling a spacecraft to precisely orient itself in desired directions. This capability is essential for vital operations such as maximizing solar power generation, optimizing antenna gain for communications, and achieving the strict alignment tolerances required for terminal docking. While numerous complex control architectures exist to manage spacecraft orientation, this work employs a Proportional-Derivative (PD) controller due to its robust reliability and straightforward implementation.

The PD controller computes the necessary control torque, $\mathbf{u}$, based on the spacecraft's current attitude and angular velocity. The control law is defined as:

$$\mathbf{u} = -K_p \boldsymbol{\theta}_e - K_d \boldsymbol{\omega}$$

where K_p and K_d represent the proportional and derivative gains, $\boldsymbol{\omega}$ is the measured angular velocity, and $\boldsymbol{\theta}_e$ is the attitude error expressed as equivalent Euler angles.

To compute $\boldsymbol{\theta}_e$ from the full nonlinear kinematics, we first calculate the error quaternion, $\delta \mathbf{q}$, using quaternion multiplication between the conjugate of the current attitude, $\mathbf{q}^*$, and

the desired reference attitude, $\mathbf{q}_{ref}$, such that $\delta\mathbf{q} = \mathbf{q}^* \otimes \mathbf{q}_{ref}$. To guarantee the shortest, most energy-efficient rotational path, the error quaternion is conditioned so that its scalar component is always positive ($\delta q_0 > 0$). If δq_0 evaluates to a negative value, the entire error quaternion is negated. Finally, leveraging the small-angle approximation, the equivalent Euler angle error is extracted directly from the vector part of the corrected error quaternion: $\boldsymbol{\theta}_e \approx 2\delta\mathbf{q}_v$.

5.2.1 Controller Design - PD Controller

To systematically determine the optimal gains, we analyze the linearized rotational dynamics of the spacecraft. For a small angular displacement, the single-axis rigid body dynamics are approximated as $J\ddot{\theta}_e = u$, where J is the principal moment of inertia. By substituting our control law directly into this dynamic model, the closed-loop system perfectly mirrors a standard second-order linear equation:

$$\ddot{\theta}_e + \frac{K_d}{J}\dot{\theta}_e + \frac{K_p}{J}\theta_e = 0 \quad \equiv \quad \ddot{\theta}_e + 2\zeta\omega_n\dot{\theta}_e + \omega_n^2\theta_e = 0$$

Equating the coefficients allows us to analytically solve for the gains based on the spacecraft's inertia (J), a desired natural frequency (ω_n), and a desired damping ratio (ζ):

$$K_p = J\omega_n^2, \quad K_d = 2\zeta\omega_n J$$

Following an iterative tuning process, the proportional and derivative gains were optimally established at $K_p = 100$ and $K_d = 1700$, respectively, yielding the most favorable system response.

5.2.2 Reaction Control System (RCS)

To actively manage the spacecraft's attitude, a 12-thruster Reaction Control System (RCS) is utilized. The architecture consists of four orthogonally mounted thruster clusters, each symmetrically positioned with a constant moment arm of $l = 1.75$ m relative to the center of mass (Figure 4). With a maximum thrust of 5 N per nozzle, an allocation matrix $\mathbf{B}_{RCS}$ is constructed to map individual thruster firings to the resulting 3D body torques via the cross product of their position ($\mathbf{r}_i$) and directional ($\mathbf{d}_i$) vectors:

$$\boldsymbol{\tau} = \mathbf{B}_{RCS}\mathbf{u} = F_{max} \begin{bmatrix} (\mathbf{r}_A \times \mathbf{d}_{A,1}) & \dots & (\mathbf{r}_D \times \mathbf{d}_{D,3}) \end{bmatrix} \mathbf{u}$$

For pure attitude control, the allocation algorithm decouples the commanded torque into roll, pitch, and yaw components, firing symmetric thruster pairs to produce pure couples. This ensures the net translational force remains zero ($\sum F = 0$), preventing unwanted orbital drift. Finally, to accurately model hardware limitations, a strict deadband of 2 Nm is enforced across all axes to prevent high-frequency chattering during minor steady-state corrections, and the pneumatic valves are restricted to a discrete operational frequency of 10 Hz.

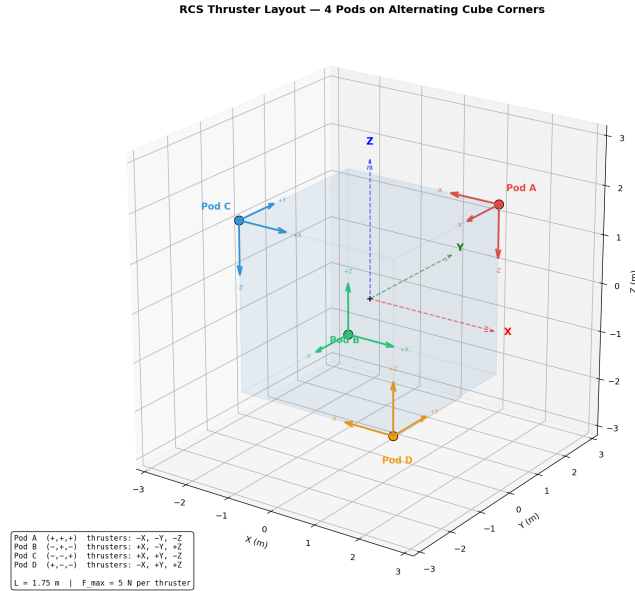


Figure 4: Geometric configuration of the 12-thruster Reaction Control System.

To evaluate the performance of the attitude control architecture and the RCS actuator model, a constant random reference quaternion, $\mathbf{q}_{ref}$, was commanded as a setpoint. The resulting closed-loop kinematic and dynamic responses are presented below.

The simulation results demonstrate successful tracking of the commanded reference quaternion. The closed-loop attitude dynamics exhibit a robust transient response, achieving a 5% settling time of approximately 130 seconds, with the principal rotation angle error converging to strictly under 1° in just 78 seconds. While a minor transient overshoot is present, it remains exceptionally small and comfortably within operational margins. Furthermore, the peak angular velocity during the maneuver reaches a maximum of only 0.0541 rad/s (approximately $3.1^\circ/\text{s}$), ensuring that rotational constraints and structural safety limits are rigorously maintained.

Examining the control effort confirms the precise operation of the RCS allocation and hardware limits. Thruster firings are strictly gated by the modeled deadband, selectively activating the corresponding rotation couples only when the required torque threshold is breached. Following the initial transient settling phase, the actuators exhibit anticipated limit-cycle behavior. As the steady-state tracking error occasionally drifts past the deadband boundary, the RCS predictably delivers brief, discrete pulses to correct the attitude, immediately deactivating once the error is restored to within the threshold constraint.

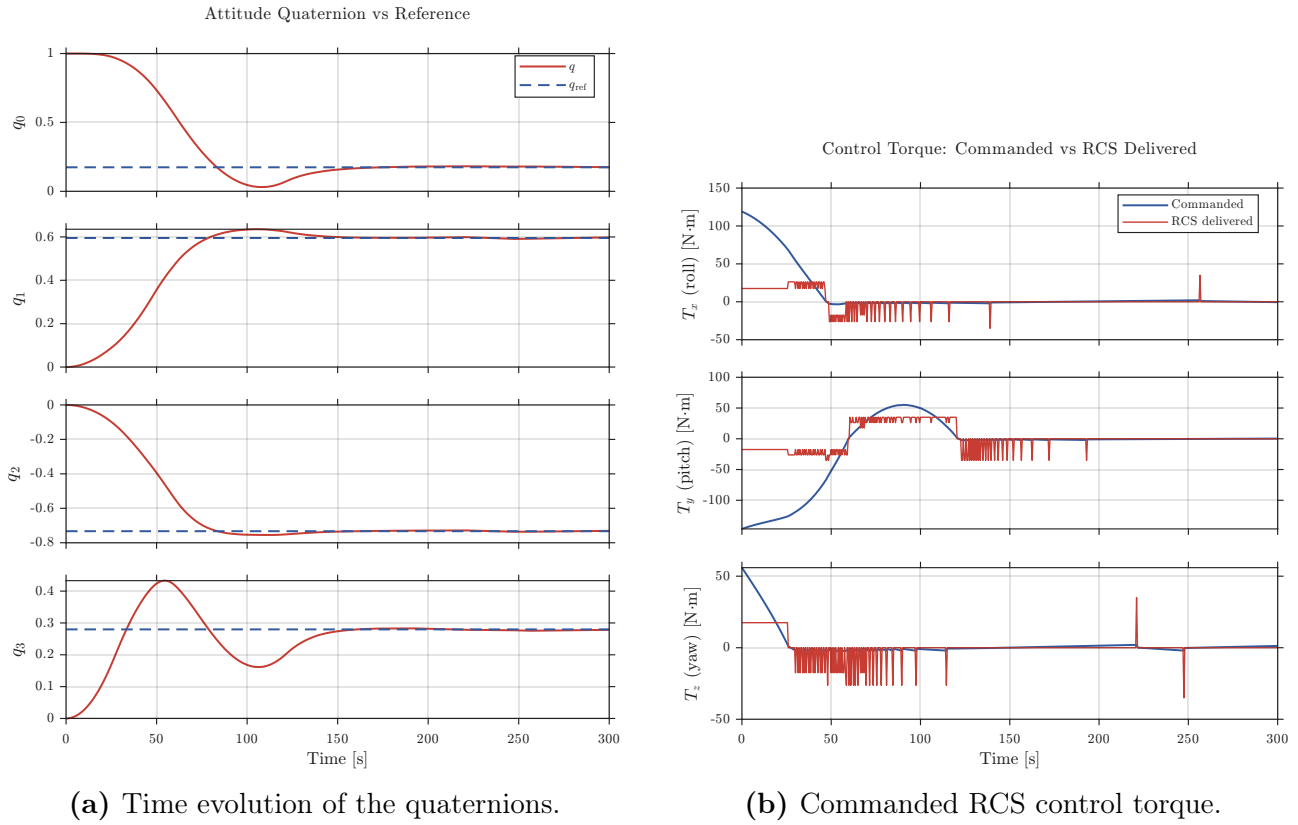


Figure 5: Attitude quaternion response and the corresponding control effort.

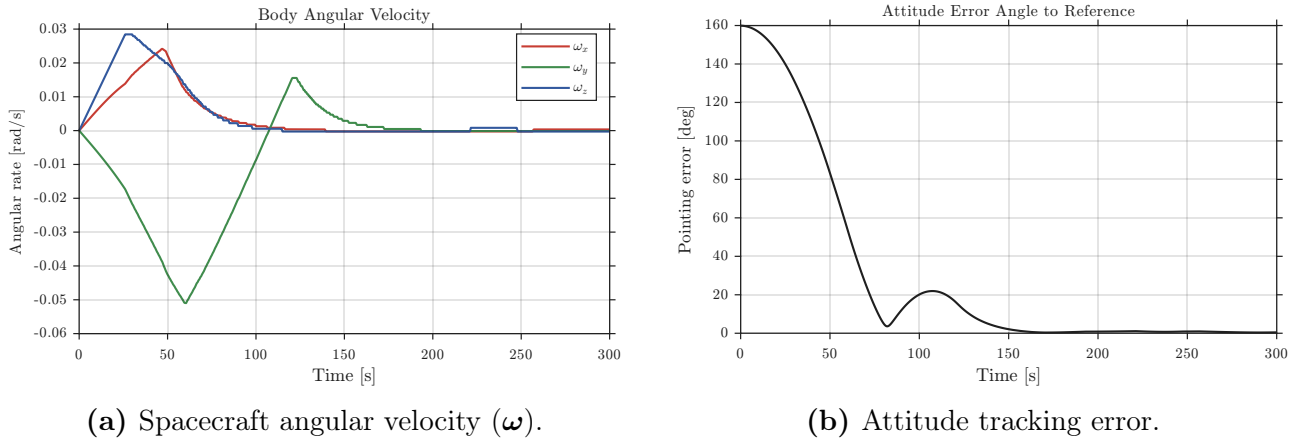


Figure 6: Angular velocity rates and the resulting attitude tracking error.

5.3 Position Estimation - KF

In a rendezvous and docking mission the controller needs to know where the chaser is relative to the target at all times. In a perfect simulation this is trivial, you just read the state directly, but once the sensors are noisy the true relative position and velocity are never exactly known, so something has to estimate them from the measurements. That is the job of the Navigation subsystem: it fuses GPS, IMU, LIDAR, and Camera into the best estimate of the 6-element relative state $\mathbf{x} = [\delta \mathbf{r}, \delta \mathbf{v}]^\top$ in the LVLH frame, which is then passed to the LQR controller.

The approach is split into two phases: Rendezvous navigation handles the long and medium range approach using GPS, IMU, and LIDAR. Docking navigation handles the final close-range phase, where a camera takes over. Both phases use the same linear Kalman Filter built around

the Clohessy–Wiltshire (CW) model; what changes between them is the active sensor and the measurement noise the filter is told to expect.

5.3.1 Reference Frame and Navigation State

All estimates use the Local Vertical Local Horizontal (LVLH) frame centred on the target. The state the filter estimates is

$$\mathbf{x}_k = [\delta r_x, \delta r_y, \delta r_z, \delta v_x, \delta v_y, \delta v_z]^\top, \quad (9)$$

each component being the chaser’s relative position or velocity along x , y , z .

The CW model needs the orbital mean motion n . For our target orbit at $h = 400$ km

$$n = \sqrt{\frac{\mu_\oplus}{(R_\oplus + h)^3}} = \sqrt{\frac{3.986 \times 10^{14}}{(6\,371\,000 + 400\,000)^3}} \approx 1.132 \times 10^{-3} \text{ rad s}^{-1}. \quad (10)$$

This is passed into the KF via the `Const_n` block and must be updated if the target altitude changes.

5.3.2 CW Process Model

The Kalman Filter needs a model of how the state evolves between steps; the CW dynamics provide it. CW gives a linear, closed-form description of relative orbital motion, so we use it as the prediction model. The filter runs at a fixed step $\Delta t = 0.01$ s, so we use the discrete-time form $\mathbf{x}_{k+1} = \Phi(\Delta t) \mathbf{x}_k + B_d \mathbf{u}_k$. With $\theta = n\Delta t$, $c = \cos \theta$, $s = \sin \theta$, the state transition matrix partitions into four 3×3 blocks:

$$\Phi_{rr} = \begin{bmatrix} 4 - 3c & 0 & 0 \\ 6(s - \theta) & 1 & 0 \\ 0 & 0 & c \end{bmatrix} \quad \Phi_{rv} = \begin{bmatrix} s/n & 2(1 - c)/n & 0 \\ -2(1 - c)/n & (4s - 3\theta)/n & 0 \\ 0 & 0 & s/n \end{bmatrix} \quad (11)$$

$$\Phi_{vr} = \begin{bmatrix} 3ns & 0 & 0 \\ 6n(c - 1) & 0 & 0 \\ 0 & 0 & -ns \end{bmatrix} \quad \Phi_{vv} = \begin{bmatrix} c & 2s & 0 \\ -2s & 4c - 3 & 0 \\ 0 & 0 & c \end{bmatrix} \quad (12)$$

The discrete control input matrix uses the constant-acceleration approximation $B_d = [\frac{1}{2}\Delta t^2 \mathbf{I}_3; \Delta t \mathbf{I}_3]^\top$. This is fine because $\Delta t = 0.01$ s is tiny next to the orbital timescale $1/n \approx 885$ s. Both Φ and B_d are rebuilt inside the KF every step from `Const_n` and `Const_dt`.

5.3.3 Sensors

Three sensors are active during rendezvous. The main tradeoff is accuracy versus range: GPS works at any distance but is noisy; LIDAR is far more precise but only useful up close.

GPS gives the chaser’s absolute position in ECI; the Sensor Manager differences the chaser and target solutions to get relative position in LVLH. We model independent zero-mean Gaussian noise with $\sigma_{\text{gps}} = 5$ m, a middle-ground value for a standalone LEO receiver (typical hardware sits between 3 and 10 m). GPS updates at 1 Hz. Simulink’s Band-Limited White Noise block takes a noise power, so $\text{power}_{\text{gps}} = \sigma_{\text{gps}}^2 \Delta t_{\text{gps}} = 25 \text{ m}^2 \text{ s}$, and a Zero-Order Hold holds the value between epochs.

IMU measures the chaser’s acceleration, used as the control input $\mathbf{u}$ in the prediction step. We model $\sigma_{\text{imu}} = 0.01 \text{ m s}^{-2}$, typical for a MEMS accelerometer on a small spacecraft, with bias assumed calibrated out. It runs at 10 Hz, so $\text{power}_{\text{imu}} = \sigma_{\text{imu}}^2 \Delta t_{\text{imu}} = 1 \times 10^{-5} \text{ m}^2 \text{ s}^{-3}$. The

IMU noise does not enter the measurement model; it feeds the process noise Q (Section 5.3.4), because a noisy accelerometer is an uncertainty in the prediction, not in a position measurement.

LIDAR activates below roughly 2 km. Its noise grows with distance, so we model

$$\sigma_{\text{lidar}}(R) = \max(\sigma_{\text{lidar,min}}, \sigma_{\text{lidar,scale}} \times R) = \max(0.02, 0.001 R), \quad (13)$$

with R the range in metres. The scale factor (0.1 % of range) is reasonable for a time-of-flight LIDAR: at 500 m this is $\sigma = 0.5$ m, already far better than GPS. The 0.02 m floor stops the uncertainty going to zero at close range (which would make the filter overconfident); it kicks in at $R = 20$ m.

5.3.4 Kalman Filter

Q represents how much we trust the prediction, and its biggest source is the IMU noise. We derive Q from the IMU so the two stay consistent:

$$Q = B_d (\sigma_{\text{imu}}^2 \mathbf{I}_3) B_d^\top = \begin{bmatrix} \frac{1}{4} \Delta t^4 \sigma_{\text{imu}}^2 \mathbf{I}_3 & \frac{1}{2} \Delta t^3 \sigma_{\text{imu}}^2 \mathbf{I}_3 \\ \frac{1}{2} \Delta t^3 \sigma_{\text{imu}}^2 \mathbf{I}_3 & \Delta t^2 \sigma_{\text{imu}}^2 \mathbf{I}_3 \end{bmatrix}. \quad (14)$$

With our values Q is tiny, which makes sense because CW is a very good description of the real dynamics.

All sensors measure position only, so $H = [\mathbf{I}_3 \quad \mathbf{0}_{3 \times 3}]$ and $\mathbf{y}_k = H\mathbf{x}_k + \mathbf{v}_k$, $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, R_k)$. The Sensor Manager sets $R_k = \sigma_{\text{active}}^2 \mathbf{I}_3$ to match whichever sensor is active, so switching from GPS to LIDAR drops R_k , raises the Kalman gain, and automatically gives more weight to the better measurement.

The filter runs at 100 Hz with the standard structure. The time update runs every step:

$$\hat{\mathbf{x}}_k^- = \Phi \hat{\mathbf{x}}_{k-1} + B_d \mathbf{u}_{k-1}, \quad P_k^- = \Phi P_{k-1} \Phi^\top + Q. \quad (15)$$

The measurement update runs only when a new measurement arrives:

$$K_k = P_k^- H^\top (H P_k^- H^\top + R_k)^{-1}, \quad \hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + K_k (\mathbf{y}_k - H \hat{\mathbf{x}}_k^-), \quad P_k = (\mathbf{I}_6 - K_k H) P_k^-. \quad (16)$$

After each update we symmetrise $P \leftarrow \frac{1}{2}(P + P^\top)$ to stop floating-point drift over many steps.

The initial covariance is $P_0 = \text{diag}(\sigma_{r0}^2 [\mathbf{1}_3], \sigma_{v0}^2 [\mathbf{1}_3])$ with $\sigma_{r0} = 50$ m, $\sigma_{v0} = 1 \text{ m s}^{-1}$. We deliberately set these larger than the GPS noise so the filter starts under-confident and trusts its first measurements; starting overconfident is much worse, as the filter would ignore data until P slowly grew back up.

5.3.5 Sensor Manager and the `update_flag`

The Sensor Manager picks which measurement feeds the KF and reports the matching R_k , based on the current estimated range. During rendezvous the handoff is: GPS only for $R > 2000$ m, then LIDAR selected for $50 \text{ m} < R \leq 2000$ m.

The KF runs at 100 Hz but GPS only updates at 1 Hz, and the ZOH holds the last value in between. If we ran the correction every step, the filter would fuse the same stale GPS reading 100 times a second, shrinking P each time as if new information had arrived, ending up overconfident and ignoring measurements. To prevent this, a Discrete Pulse Generator fires a 1 Hz pulse that sets `update_flag`; the correction equations only run when `update_flag` > 0.5, otherwise the filter passes the prediction straight through ($\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^-$, $P_k = P_k^-$).

5.4 Attitude Estimation - MEKF

5.5 Sensors

5.5.1 Star Trackers

The star sensor determines the satellite's triaxial attitude to the celestial coordinate system (inertial system) by detecting the radiation from the star [4].

These trackers autonomously match the tracked stars with stars in an internal catalogue and use a certain method to compute the star tracker's attitude with respect to a celestial reference frame. A typical tracker has an update rate between 0.5 and 10 Hz, a mass of about 3 kg and a power requirement on the order of 10 W [5].

A star tracker is basically a digital camera with a focal plane populated by pixels (picture elements).

A star tracker has two modes of operation: *tracking mode*, in which the tracker is following several stars that have already been matched with catalogued stars, and *initial attitude acquisition*, in which the tracker searches the entire field of view (FOV) for the brightest clusters of pixels and computes at least three centroids [5].

Our code implements a realistic simulation of sensor measurement noise for a Star Tracker. In simple terms, it takes the satellite's perfect, actual attitude (the "true" quaternion) and corrupts it by adding a three-dimensional random error. This simulates the hardware imperfections, optical distortions, and electronic noise that occur in a real sensor.

Listing 1: Star Tracker noise model

```
1 sigma = 9.7e-5;
2 theta_err = sigma * randn(3, 1);
```

This represents the uncertainty of the star tracker along its three principal axes (roll, pitch, and yaw). A value of 9.7×10^{-5} rad is equal to approximately 20 arcseconds, which is a standard, highly realistic accuracy envelope for commercial space-grade star trackers.

In 3D space, rotational errors cannot simply be added together linearly because 3D rotations are non-commutative (the order of rotation matters). Instead, we must use quaternions. The block below uses the standard mathematical axis-angle formula to transform a small 3D angular displacement vector into its equivalent quaternion representation.

```
1 mag_err = norm(theta_err);
2
3 if mag_err > 1e-10
4     dq = [cos(mag_err/2); (sin(mag_err/2)/mag_err) * theta_err];
5 else
6     dq = [1; 0; 0; 0];
7 end
```

To combine or apply successive rotations using quaternions, you must multiply them. Mathematically, this line executes the Hamilton product:

$$\mathbf{q}_{\text{meas}} = \delta \mathbf{q} \otimes \mathbf{q}_{\text{true}}$$

```
1 w1 = dq(1);    v1 = dq(2:4);
2 w2 = q_true(1); v2 = q_true(2:4);
3
4 q_meas = [w1*w2 - dot(v1, v2);
5           w1*v2 + w2*v1 + cross(v1, v2)];
```

The resulting $\mathbf{q}_{\text{meas}}$ is the perturbed (noisy) attitude. This is the actual value that the star tracker would measure and send to the satellite's On-Board Computer (OBC).

For a quaternion to represent a valid physical rotation in space, it must strictly be a unit quaternion (its norm must equal exactly 1). Due to tiny floating-point rounding errors inherent to computer processors, the norm might slightly drift (e.g., becoming 1.000001).

This final step ensures mathematical consistency before exporting the variable.

```
1 q_meas = q_meas / norm(q_meas);
```

5.5.2 Gyroscopes

There are different types of gyroscopes that have different measuring principles, but they can all be used to measure the angular velocity of a spacecraft relative to inertial space [4].

The classical spinning-mass gyros depend on the tendency of the angular momentum of a rotating mass to remain fixed in inertial space. These are used in torque rebalance mode, meaning that the angular momentum is held constant in the spacecraft frame. The torque required to keep the gyro's angular momentum $\mathbf{H}_B$ constant is determined by:

$$\mathbf{L}_B = \boldsymbol{\omega}_{BI}^B \times \mathbf{H}_B$$

The two components of $\boldsymbol{\omega}_{BI}^B$ along the input axes perpendicular to $\mathbf{H}_B$ determine the required two components of $\mathbf{L}_B$ along the output axes, given by the cross products of the input axes with the direction of $\mathbf{H}_B$ [5].

A widely used three-axis continuous-time mathematical model for a rate-integrating gyro is given by:

$$\boldsymbol{\omega} = \boldsymbol{\omega}_{\text{true}} + \boldsymbol{\beta}_{\text{true}} + \boldsymbol{\eta}_v \quad (17)$$

$$\dot{\boldsymbol{\beta}}_{\text{true}} = \boldsymbol{\eta}_u \quad (18)$$

where $\boldsymbol{\omega}_{\text{true}}$ is the true rate, $\boldsymbol{\omega}$ is the measured rate, $\boldsymbol{\beta}_{\text{true}}$ is the true bias or drift, and $\boldsymbol{\eta}_v$ and $\boldsymbol{\eta}_u$ are independent zero-mean Gaussian white-noise processes [5].

In our model, we use the Three-Axis Gyroscope block that implements a gyroscope on each of the three axes.

Input:

- $\boldsymbol{\omega}$: angular rates in the body-fixed axes, specified as a three-element vector, in rad/s.

Output:

- $\boldsymbol{\omega}_{\text{meas}}$: measured angular rates from the gyroscope, returned as a three-element vector, in rad/s.

Parameters:

- Natural frequency of the gyroscope: 190 rad/s.
- Damping ratio: 0.707.
- Measurement bias: [0.01, -0.005, 0.02] rad/s.
- Noise power (PSD): [10^{-8} , 10^{-8} , 10^{-8}] (rad/s)²/Hz.

5.5.3 Laser Radar

A laser radar is a radar using laser as the carrier because the laser provides optical-waveband electromagnetic radiation, whose wavelength is much shorter than microwave and millimetre wave. A laser radar requires cooperative target angle reflectors on the target to reflect the laser back to the laser radar [4].

It has a series of unique advantages: high angular resolution, high range resolution, high speed resolution, wide range of velocity measurement, ability to obtain a variety of target images, and strong anti-interference power. At the same time, the volume and mass of the laser radar are lower than those of the microwave radar [4].

This code simulates how the LiDAR measures the orientation of the chaser relative to the target, including the sensor's physical measurement limitations and noise.

Listing 2: Relative attitude computation

```

1 w_i = q_target(1); v_i = -q_target(2:4);
2 w_c = q_chaser(1); v_c = q_chaser(2:4);
3
4 q_rel = [ w_i*w_c - dot(v_i, v_c);
5           w_i*v_c + w_c*v_i + cross(v_i, v_c)];

```

It changes the frame of reference, answering the fundamental question: “*If my Laser Radar is looking at the target’s docking port, what is the geometric angular offset between the chaser’s body frame and the target’s body frame?*” The inversion (conjugate) of $\mathbf{q}_{\text{target}}$ subtracts the target’s absolute movement, so only the relative attitude remains.

Listing 3: LiDAR noise model

```

1 sigma = 5e-4;
2 theta_err = sigma * randn(3,1);
3 mag_err = norm(theta_err);
4
5 if mag_err > 1e-10
6     dq = [cos(mag_err/2); (sin(mag_err/2)/mag_err) * theta_err];
7 else
8     dq = [1; 0; 0; 0];
9 end

```

Unlike a star tracker which looks at distant stars, a Laser Radar interacts with a nearby physical object. Its accuracy depends on laser pulse timing, spot-centre estimation on the retroreflectors, and structural vibrations.

A standard deviation of $\sigma = 5 \times 10^{-4}$ rad (approx. 100 arcseconds) is a highly accurate representation of a coarse rendezvous LiDAR at close-to-medium ranges. The code samples this uncertainty across three orthogonal axes using a Gaussian distribution (`randn`), capturing the physical jitter of the laser reflections, and then converts this 3D angular jitter into a small error quaternion $\delta\mathbf{q}$.

```

1 w1 = dq(1); v1 = dq(2:4);
2 w2 = q_rel(1); v2 = q_rel(2:4);
3
4 q_rel_meas = [ w1*w2 - dot(v1, v2);
5               w1*v2 + w2*v1 + cross(v1, v2) ];

```

This combines the true relative docking alignment with the high-frequency measurement noise of the laser optics. The output $\mathbf{q}_{\text{rel, meas}}$ is the actual “noisy” telemetry data sent to the docking computer.

The Star Tracker (1 Hz) prioritizes maximum absolute precision over the long term, accepting a slower update rate due to optical exposure times and complex star identification algorithms. The LiDAR (10 Hz) prioritizes real-time responsiveness and safety during critical short-range maneuvers, where the proximity of the spacecraft demands significantly faster control loops to prevent collisions.

5.6 MEKF

The basic idea of the Multiplicative Extended Kalman Filter (MEKF) is to use the quaternion as the “global” attitude representation and use a three-component state vector $\delta\boldsymbol{\theta}$ for the “local” representation of attitude errors. Instead of writing the true quaternion as the sum of the estimated quaternion and an error quaternion, we write it as the product of an error quaternion [5].

We use the MEKF because it handles the 3D geometry of space rotations correctly: it propagates the full 4D quaternion while maintaining its unit-norm constraint, while restricting the Kalman filtering process to a minimal 3D representation of the angular error, ensuring maximum numerical stability and accuracy.

The MEKF updates the error state:

$$\delta\mathbf{x} = \begin{bmatrix} \delta\boldsymbol{\theta} \\ \delta\boldsymbol{\xi} \end{bmatrix}$$

where $\delta\boldsymbol{\theta}$ represents the attitude error and $\delta\boldsymbol{\xi}$ represents the bias error of the gyroscope.

Our attitude filters all use some variant of the MEKF, which proceeds by iteration of three steps:

- **Measurement update:** updates the error state vector.
- **State vector reset:** moves the updated information from the error state to the global attitude representation and resets the components of the error state to zero.
- **Propagation:** propagates the global variables to the time of the next measurement.

5.6.1 Measurement Update

The observation model for attitude estimation is:

$$\mathbf{z}_k = h(\mathbf{x}_k) + \boldsymbol{\nu}_k$$

The MEKF updates the error state, not the global state $\mathbf{q}$ and $\boldsymbol{\xi}$, so the measurement sensitivity matrix is:

$$H = \left. \frac{\partial h}{\partial \delta\mathbf{x}} \right|_{\delta\mathbf{x}=0}$$

The error quaternion to first order in the error vector is:

$$\delta\mathbf{q} \approx \begin{bmatrix} 1 \\ \frac{1}{2}\delta\boldsymbol{\theta} \end{bmatrix}$$

Using the Rodrigues vector parameterisation, the measurement used in the filter is:

$$\mathbf{z}_k = 2\delta\mathbf{q}_{1:3}$$

The measurement sensitivity matrix is:

$$H = \begin{bmatrix} I_{3 \times 3} & 0_{3 \times 3} \end{bmatrix}$$

The state update for the k th measurement (after a reset) simplifies to:

$$\delta \hat{\mathbf{x}}_k^+ = K_k \mathbf{r}_k$$

The Kalman gain is:

$$K = PH^\top (HPH^\top + R)^{-1}$$

where H is the sensitivity matrix, R is the covariance matrix of measurement noise, and P is the covariance matrix of the state error [5].

Listing 4: Kalman gain and update

```

1 q_upd_inv = [q_upd(1); -q_upd(2:4)];
2 q_err = quat_mult(q_upd_inv, q_meas);
3 res = 2 * q_err(2:4);
4
5 H = [eye(3), zeros(3,3)];
6 S = H * P * H' + R_total;
7 K = (P * H') / S;
8 dx = K * res;
```

5.6.2 Reset

After the measurement update, the reset procedure moves the information to the post-update global state $\mathbf{q}^+$ and $\boldsymbol{\xi}^+$ while resetting $\delta\boldsymbol{\theta}$ and $\delta\boldsymbol{\xi}$ to zero. The reset must obey:

$$\mathbf{q}^+ = \delta\mathbf{q}(\delta\boldsymbol{\theta}^+) \otimes \mathbf{q}^-, \quad \boldsymbol{\xi}^+ = \boldsymbol{\xi}^- + \delta\boldsymbol{\xi}^+$$

Listing 5: Reset step

```

1 d_theta = dx(1:3);
2 mag_dtheta = norm(d_theta);
3
4 if mag_dtheta > 1e-12
5     dq_update = [cos(mag_dtheta/2); (d_theta/mag_dtheta)*sin(mag_dtheta/2)];
6 else
7     dq_update = [1; 0; 0; 0];
8 end
9
10 q_upd = quat_mult(q_upd, dq_update);
11 beta_upd = beta_upd + dx(4:6);
```

5.6.3 Propagation

The estimated quaternion satisfies the kinematic equation:

$$\dot{\mathbf{q}} = \frac{1}{2} \Xi(\mathbf{q}) \hat{\boldsymbol{\omega}}$$

where $\hat{\boldsymbol{\omega}} = \boldsymbol{\omega}_{\text{meas}} - \hat{\boldsymbol{\beta}}$.

Listing 6: Quaternion propagation

```

1 w_hat = w_meas - beta_est;
2 mag_w = norm(w_hat);
```

```

3
4 if mag_w > 1e-12
5     dq_prop = [cos(0.5 * mag_w * dt);
6                 (w_hat / mag_w) * sin(0.5 * mag_w * dt)];
7 else
8     dq_prop = [1; 0; 0; 0];
9 end
10
11 q_minus = quat_mult(q_in, dq_prop);

```

Updating the quaternion from step k to step $k+1$ is achieved by computing the incremental rotation quaternion $\delta \mathbf{q}_{\text{prop}}$; the final integration reduces to a simple quaternion multiplication.

The continuous error-state vector obeys the linearized dynamic equation:

$$\dot{\delta \mathbf{x}} = F(t) \delta \mathbf{x} + G(t) \boldsymbol{\eta}$$

A discrete-time propagation can be used for the covariance matrix. For small dt , adequate results are provided by an even simpler first-order approximation:

$$\Phi_k \approx I_n + \Delta t F(t)$$

$$Q_k \approx \Delta t G Q G^\top$$

The matrices $F(t)$, $G(t)$ are given by:

$$F(t) = \begin{bmatrix} -[\hat{\boldsymbol{\omega}}(t) \times] & -I_3 \\ 0_{3 \times 3} & 0_{3 \times 3} \end{bmatrix}, \quad G(t) = \begin{bmatrix} -I_3 & 0_{3 \times 3} \\ 0_{3 \times 3} & I_3 \end{bmatrix}$$

We get the state transition matrix:

$$\Phi = \begin{bmatrix} I_{3 \times 3} & 0_{3 \times 3} \\ 0_{3 \times 3} & I_{3 \times 3} \end{bmatrix} + \begin{bmatrix} -[\hat{\boldsymbol{\omega}} \times] & -I_{3 \times 3} \\ 0_{3 \times 3} & 0_{3 \times 3} \end{bmatrix} \cdot dt$$

$$\Phi = \begin{bmatrix} I_{3 \times 3} - [\hat{\boldsymbol{\omega}} \times] dt & -I_{3 \times 3} dt \\ 0_{3 \times 3} & I_{3 \times 3} \end{bmatrix}$$

The covariance matrix P represents the uncertainty of the state. As time progresses by an interval dt , the previous uncertainty P_k is transformed through the dynamics of the system using the discrete state transition matrix Φ . To this updated uncertainty, we add the new process noise Q accumulated over the interval dt :

$$P_{k+1} = \Phi P_k \Phi^\top + Q_k$$

Listing 7: Covariance propagation

```

1 W_skew = [      0,    -w_hat(3),    w_hat(2);
2            w_hat(3),      0,    -w_hat(1);
3            -w_hat(2),    w_hat(1),      0 ];
4
5 Phi = [eye(3) - W_skew*dt,  -eye(3)*dt;
6         zeros(3,3),         eye(3)  ];
7
8 P = Phi * P * Phi' + Q_noise * dt;

```

6 Terminal Docking

During the Terminal Docking phase the chaser transitions from the Near-Field hold-point ($\approx \text{m}$) to physical contact with the target docking port. This phase demands a more sophisticated controller than the LQR used in the Near-Field phase for three reasons: (i) the MPC can plan a smooth deceleration profile over a long predictive horizon; (ii) it enforces hard actuator constraints directly inside the optimiser; and (iii) a time-varying waypoint sequencer generates a coupled lateral– radial reference trajectory, something a fixed-setpoint LQR cannot do.

6.1 Docking Port Geometry

Both spacecraft expose a single docking port defined by a position offset from the centre of mass (CoM) and an orientation quaternion, all expressed in the body frame ($\mathbf{B}$):

Parameter	Target	Chaser
Port offset $\mathbf{p}^b$	$[-15, 0, 0]^\top \text{ m}$	$[+2, 0, 0]^\top \text{ m}$
Port quaternion $\mathbf{q}_{\text{port}}^b$	$[0, 0, 0, 1]^\top$ (180° about z)	$[1, 0, 0, 0]^\top$ (identity)
Port faces	$-x_b$	$+x_b$

The chaser CoM must arrive at the contact point derived by aligning the two port faces along the R-bar (x_{LVH}) axis:

$$x_{\text{dock}} = p_{\text{tgt},x}^b - p_{\text{chs},x}^b = -15 - 2 = -17 \text{ m} \quad (19)$$

A 10-cm safety stop is implemented at $x_{\text{hold}} = -16.9 \text{ m}$; the simulation terminates when $\|\mathbf{r}_{\text{rel}} - \mathbf{x}_{\text{dock}}\| \leq 0.1 \text{ m}$ (verified by a *Compare to Constant* ≤ 0.1 block connected to a *Stop Simulation* block in the Simulink model).

6.2 MPC Controller

6.2.1 Discrete-Time Model

The MPC uses the same CW linearisation as the LQR, discretised via ZOH at $T_s = 0.2 \text{ s}$:

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k, \quad \mathbf{x}_k = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^\top, \quad \mathbf{u}_k = [F_x, F_y, F_z]^\top \quad (20)$$

with mean motion $n = \sqrt{\mu_\oplus / (R_\oplus + h)^3} \approx 1.1332 \times 10^{-3} \text{ rad s}^{-1}$ (hardcoded as `Const_n` in the Simulink KF subsystem).

6.2.2 Optimisation Problem

At every sample step the MPC block solves:

$$\min_{\mathbf{u}_0, \dots, \mathbf{u}_{N_c-1}} \sum_{k=0}^{N_p} [(\mathbf{x}_k - \mathbf{x}_{\text{ref}})^\top \mathbf{Q} (\mathbf{x}_k - \mathbf{x}_{\text{ref}}) + \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k + \Delta \mathbf{u}_k^\top \mathbf{R}_{\Delta u} \Delta \mathbf{u}_k] \quad (21)$$

with $N_p = 100$ (20 s lookahead) and $N_c = 30$ (6 s of authority) at $T_s = 0.2 \text{ s}$.

6.2.3 Actuator Limits and Scaling

The maximum specific force is 0.001 m s^{-2} , giving for $m = 10\,000 \text{ kg}$:

$$F_{\max} = m \cdot a_{\max} = 10\,000 \times 0.001 = 10 \text{ N per axis} \quad (22)$$

Scale factors normalise the QP variables to order unity:

Channel	Scale factor	Rationale
Position outputs (OV 1–3)	20 m	port at -17 m , bound is 20 m
Velocity outputs (OV 4–6)	0.15 m s^{-1}	sequencer hard cap
Thrust inputs (MV 1–3)	$F_{\max} = 10 \text{ N}$	

6.2.4 Cost Function Weights

Weights are tuned to prioritise lateral station-keeping and to penalise rapid lateral thrust reversals:

$$\mathbf{Q}_{\text{diag}} = [1, 10, 10, 0.7, 250, 250], \quad \mathbf{R}_{\text{diag}} = [0.1, 0.1, 0.1], \quad \mathbf{R}_{\Delta u, \text{diag}} = [0.1, 2, 2] \quad (23)$$

The y/z velocity weights (250) are $357\times$ larger than the radial velocity weight (0.7), reflecting the tight lateral alignment requirement. An internal process-noise model $\mathbf{N} = \text{diag}(0.01, 0.01, 0.01, 5, 5, 5)$ is specified via `mpcobj.Model.Noise` to account for higher velocity uncertainty. A soft-constraint penalty $\rho_\varepsilon = 10^4$ recovers from transient infeasibility at the LQR-to-MPC hand-off.

6.3 Waypoint Sequencer

Rather than a fixed terminal state, a dedicated MATLAB Function block (`waypoint_sequencer`) computes a time-varying 6-element reference at every sample step.

Radial velocity reference (glideslope law).

$$V_{x, \text{ref}} = k_g \Delta x + v_{\text{dock}}, \quad \Delta x = \max(x_{\text{dock}} - x, 0) \quad (24)$$

with $k_g = 0.002 \text{ s}^{-1}$ and $v_{\text{dock}} = 0.02 \text{ m s}^{-1}$.

Lateral velocity reference (cross-coupling). The sequencer actively commands lateral velocities to null lateral errors:

$$V_{y, \text{ref}} = -V_{x, \text{ref}} \frac{y}{\Delta x_{\text{eff}}}, \quad V_{z, \text{ref}} = -V_{x, \text{ref}} \frac{z}{\Delta x_{\text{eff}}}, \quad \Delta x_{\text{eff}} = \max(\Delta x, 1.0) \quad (25)$$

Position reference (distance-based carrot). A proportional lead distance advances the reference point ahead of the chaser. To maintain a strong closing velocity at long range while preventing integrator windup at close range, the lead distance scales linearly but is floored at a minimum of 5 m:

$$d_{\text{lead}} = \max\left(30 \frac{\Delta x}{50}, 5.0\right), \quad x_{\text{ref}} = \min(x + d_{\text{lead}}, x_{\text{hold}}) \quad (26)$$

Lateral position references scale proportionally to the remaining radial distance:

$$y_{\text{ref}} = y \frac{x_{\text{hold}} - x_{\text{ref}}}{\Delta x}, \quad z_{\text{ref}} = z \frac{x_{\text{hold}} - x_{\text{ref}}}{\Delta x} \quad (27)$$

ensuring they converge to zero exactly when x_{ref} reaches $x_{\text{hold}} = -16.9 \text{ m}$.

6.4 Attitude Controller

6.4.1 PD Control Law

The nonlinear PD controller is identical in structure to the Near-Field phase (§5.2), damping during close-range docking:

$$\mathbf{u} = -K_p \boldsymbol{\theta}_e - K_d \boldsymbol{\omega}, \quad K_p = 100, \quad K_d = 1700 \quad (28)$$

The Simulink *Pointing Controller* subsystem implements first-order filters ($\tau = 0.5$ s) on each axis of the attitude error before applying the proportional gain, smoothing out high-frequency measurement noise from the MEKF.

6.4.2 Pointing Reference

While the baseline docking reference quaternion $\mathbf{q}_{\text{ref,dock}} = [1, 0, 0, 0]^\top$ aligns the chaser with the LVLH frame at a given instant, the LVLH frame is not strictly inertial; it pitches continuously around the Earth at the orbital mean motion n .

If a static quaternion were used as the inertial reference, the chaser would hold its absolute orientation in space while the target docking port (fixed to the rotating LVLH frame) slowly pitched away from it. To maintain continuous face-to-face alignment, the reference quaternion must be actively propagated over time at the orbital rate.

Assuming the standard LVLH convention where the orbit normal aligns with the body y -axis, the constant orbital angular velocity vector is defined as:

$$\boldsymbol{\omega}_{\text{orb}} = \begin{bmatrix} 0 \\ -n \\ 0 \end{bmatrix} \quad (29)$$

Instead of altering the derivative term of the PD controller, this constant angular velocity is fed directly into an *Angular Velocity to Quaternion* integration block. Starting from the initial identity quaternion $\mathbf{q}(0) = [1, 0, 0, 0]^\top$, the block continuously solves the standard quaternion kinematic equation:

$$\dot{\mathbf{q}}_{\text{ref}} = \frac{1}{2} \mathbf{q}_{\text{ref}} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega}_{\text{orb}} \end{bmatrix} \quad (30)$$

This produces a continuously updating $\mathbf{q}_{\text{ref}}$ that natively tracks the pitching motion of the orbit. By generating the attitude error against this rotating reference, the PD controller inherently commands the chaser to pitch synchronously with the target, maintaining perfect port alignment without fighting the orbital kinematics.

6.5 RCS During Terminal Docking

Translation commands from the MPC are routed through the same 12-thruster RCS described in 5.2.2. Inside the *Simple RCS* Simulink subsystem, force commands are first normalised by F_{max} , then allocated to individual thruster pairs via the MATLAB Function `control(Tcmd, Fcmd)`, and finally modulated by three *PWM* blocks - one per body axis — with force threshold 5 N and torque deadband 2 Nm (unchanged from the Near-Field phase).

6.6 Docking Navigation

The docking phase reuses the same filter; only the sensor and the measurement noise change as the chaser closes in.

6.6.1 Camera Sensor

Below about 50 m a camera takes over, tracking known features on the docking port. At this range accuracy is roughly constant regardless of exact distance, so we use a fixed model $\sigma_{\text{cam}} = 0.03 \text{ m}$ (3 cm). This is slightly worse than the LIDAR floor at the same distance, which is expected: the camera has no precise ranging pulse, so it is a bit noisier than LIDAR.

6.6.2 Filter Behaviour at Docking

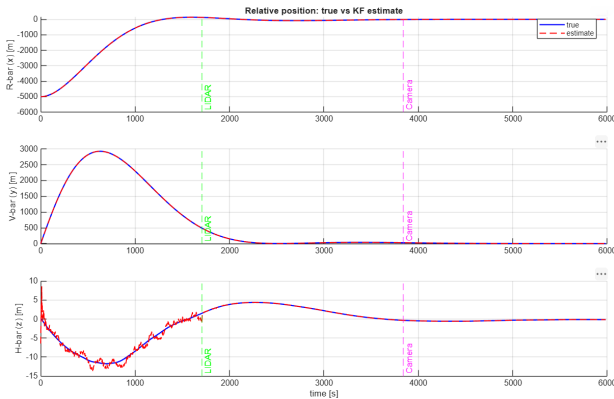
When the estimated range drops below 50 m, the Sensor Manager switches the active measurement from LIDAR to Camera and sets $R_k = \sigma_{\text{cam}}^2 \mathbf{I}_3$. Because the filter is already converged by this point (it has been running on GPS then LIDAR for the whole approach), the camera handoff is smooth: P_k^- is already small, so the gain stays moderate and the estimate simply tightens further. No structural change to the KF is needed, which is the advantage of keeping a single linear filter across all phases. Table 4 summarises all four sensors.

Table 4: Navigation sensor parameters.

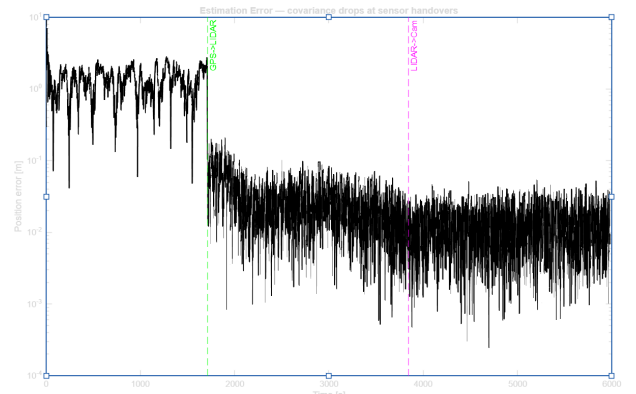
Sensor	σ	Update rate	Active range	Notes
GPS	5 m	1 Hz	All ranges	Always on
IMU	0.01 m s^{-2}	10 Hz	All ranges	Process noise only
LIDAR	$\max(0.02, 0.001R)$	10 Hz	$R \lesssim 2 \text{ km}$	Range-dependent
Camera	0.03 m	10 Hz	$R \lesssim 50 \text{ m}$	Fixed accuracy

6.6.3 Results

Figure 7 shows the true vs. estimated relative position and the estimation error over the full 6000 s mission, with LIDAR activating at $t = 1711 \text{ s}$ and Camera at $t = 3842 \text{ s}$. The radial and along track axes track almost perfectly throughout, because the GPS noise ($\pm 5 \text{ m}$) is small next to the kilometre-scale motion there and CW is a very good predictor. The cross-track axis is more telling: in the GPS-only phase the estimate carries visible $\pm 2 \text{ m}$ noise, which drops clearly once LIDAR activates and becomes essentially smooth after the camera takes over. The error stays bounded and shrinks at each handoff: cross-track error is up to $\pm 3 \text{ m}$ on GPS, below $\pm 0.5 \text{ m}$ with LIDAR, and below $\pm 0.05 \text{ m}$ with the camera; radial and along-track stay below 1 m throughout.



(a) True (blue) vs estimated (red dashed).



(b) Error $\hat{\mathbf{r}} - \mathbf{r}_{\text{true}}$.

Figure 7: Full-mission navigation results in LVLH. Green dashed: LIDAR activation ($t = 1711 \text{ s}$); magenta dashed: Camera activation ($t = 3842 \text{ s}$).

6.7 Simulation Results

The performance of the Terminal Docking GNC architecture is validated through the closed-loop simulation of the final approach. The system response is evaluated across the translational kinematics and attitude tracking.

The position response confirms the effectiveness of the proportional waypoint sequencer. The radial position along the R-bar (x) smoothly converges toward the structural contact point at -17.0 m, successfully arresting at the predefined hold point ($x_{\text{hold}} = -16.9$ m). The lateral position components (y and z) are continuously regulated throughout the approach, strictly converging to zero. This demonstrates that the distance-based carrot logic successfully guides the chaser to perfect face-to-face port alignment without integrator windup or overshoots.

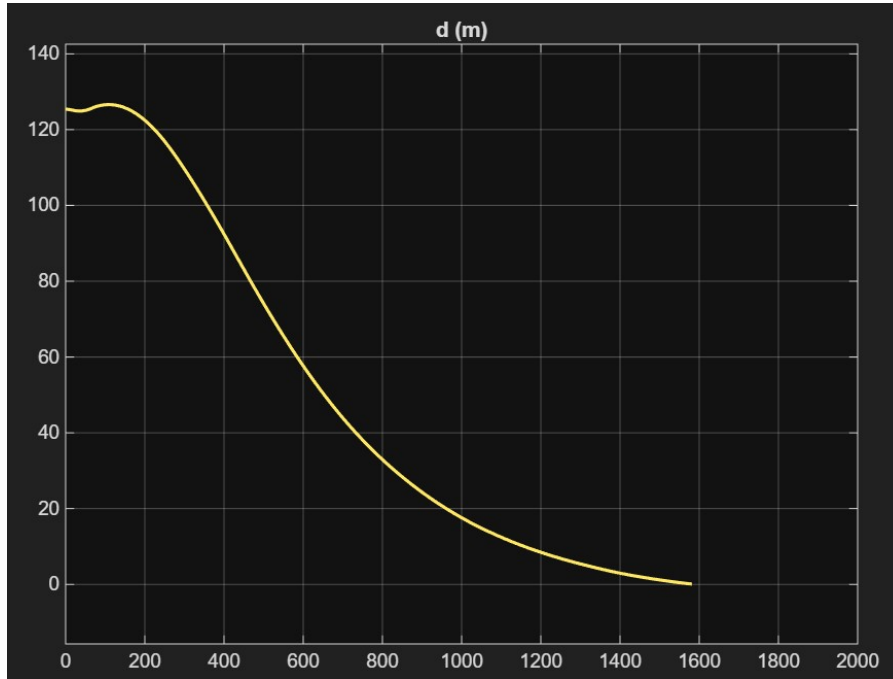


Figure 8: Relative position trajectory in the LVLH frame during terminal docking. The chaser smoothly converges along the R-bar (x -axis), arresting perfectly at the 10cm hold point with zero lateral offset.

The velocity plots demonstrate the MPC’s ability to seamlessly track the kinematic glideslope. The radial closing velocity (V_x) exhibits a steady, proportional deceleration as the chaser nears the target, reliably settling at the prescribed soft-docking terminal velocity of 0.02 m s^{-1} upon final contact. Meanwhile, the lateral velocities (V_y and V_z) undergo bounded transient corrections before gracefully tapering to zero, validating the sequencer’s cross-coupling logic designed to nullify lateral drift during the critical final meters.

The attitude quaternion evolution highlights the stability of the active rotational reference tracking. The chaser’s actual attitude smoothly aligns with the continuously propagating reference quaternion without steady-state lag or erratic thruster chattering. By natively tracking the n -rate pitch of the LVLH frame through the continuous integration block, the PD controller ensures robust, synchronous face-to-face alignment throughout the entire docking sequence.

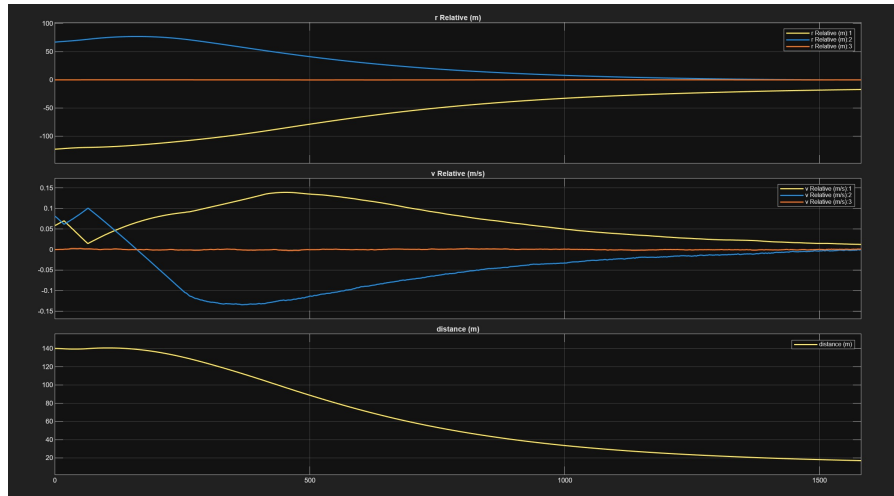


Figure 9: Relative velocity profile. The MPC perfectly executes the glideslope deceleration down to 0.02 m s^{-1} while continuously suppressing lateral drift (V_y, V_z).

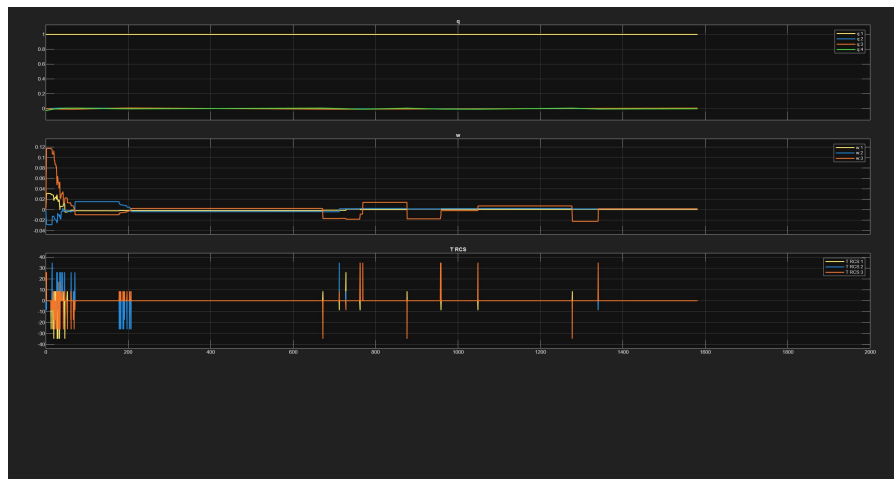


Figure 10: Attitude quaternion evolution. The chaser synchronously pitches with the LVLH frame, perfectly tracking the continuous rotational reference to maintain constant face-to-face port alignment.

References

- [1] Wigbert Fehse. *Automated Rendezvous and Docking of Spacecraft*. Cambridge University Press, 2003.
- [2] Rudolf Emil Kalman. Contributions to the theory of optimal control. *Boletín de la Sociedad Matemática Mexicana*, 5(2):102–119, 1960.
- [3] Ensatellite. Hill’s equations: Derivation and solution. <https://ensatellite.com/hills-equations/>, n.d. Accessed: 2026-06-06.
- [4] Tao Liu Yongchun Xie, Changqing Chen and Min Wang. *Guidance, Navigation, and Control for Spacecraft Rendezvous and Docking: Theory and Methods*. Springer, 2021.
- [5] F. Landis Markley and John L. Crassidis. *Fundamentals of Spacecraft Attitude Determination and Control*. Springer, 2014.